

src 源码逐行讲解

适合：会一点 Java/C，但 Python 语法几乎忘了；希望每一行都能知道 ‘它在做什么、为什么需要它’ 。

约束：本 PDF 只读取现有 src 代码生成，不修改项目源码，不读取 .env，不调用 DeepSeek API。

建议阅读顺序

1. `__init__.py`: 认识 Python 包。2. `types.py`: 认识项目里的数据对象。3. `store.py`: 理解 Session 与长期记忆。4. `tools.py`: 理解工具注册。5. `llm.py`: 理解模型通信。6. `tasks.py`: 理解异步。7. `runtime.py`: 最后阅读 Agent Loop。

Python 和 Java/C 的三个关键区别

- (1) Python 用缩进表示代码块，不用大括号。
- (2) 变量通常不用提前声明类型，`x = 3` 就能使用；类型标注多是提示。
- (3) `self` 相当于 Java 的 `this`，表示当前对象。

每行的阅读格式

灰色框是原始代码和行号；其后 ‘白话解释’ 紧跟这行代码。空行也会标出，用于说明为什么代码分段。

__init__.py

最短的文件。它把 src 标记为一个 Python 包，让其他文件能够写 `from src.runtime import ...`。

逐行开始

```
1 """Minimal Agent runtime package."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、`help()` 等工具读取。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

types.py

先定义全项目都要用的数据形状。它类似 Java 的 DTO/POJO，或 C 里约定好的 struct。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 from dataclasses import dataclass, field
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 from typing import Any
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
6 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
7 @dataclass
```

白话解释：装饰器：让 Python 自动生成构造函数等常见方法。效果接近 Java 中简化 POJO 的工具。

```
8 class ToolCall:
```

白话解释：定义类 ToolCall。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
9     id: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
10     name: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
11     arguments: dict[str, Any]
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
12 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
13 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
14 @dataclass
```

白话解释：装饰器：让 Python 自动生成构造函数等常见方法。效果接近 Java 中简化 POJO 的工具。

```
15 class LLMReply:
```

白话解释：定义类 LLMReply。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
16     content: str | None = None
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
17     tool_calls: list[ToolCall] = field(default_factory=list)
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
18 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

19 <空行>

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

20 @dataclass

白话解释：装饰器：让 Python 自动生成构造函数等常见方法。效果接近 Java 中简化 POJO 的工具。

21 class AgentResult:

白话解释：定义类 AgentResult。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

22 session_id: str

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

23 answer: str

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

24 trace: list[dict[str, Any]]

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

25 stopped_by_limit: bool = False

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

26 <空行>

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

store.py

本地数据层。负责把 session、长期记忆、待办、事件保存到 JSON 文件。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 import json
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 from pathlib import Path
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 from threading import RLock
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
6 from typing import Any
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
7 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
8 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
9 class JsonStore:
```

白话解释：定义类 JsonStore。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
10     """Inspectable local persistence. Every session key is scoped by user and session."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、help() 等工具读取。

```
11 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
12     def __init__(self, path: str | Path = "data/agent_state.json") -> None:
```

白话解释：构造函数：创建对象时自动执行，用来准备这个对象后续工作需要的状态。

```
13         self.path = Path(path)
```

白话解释：把传入的路径字符串包装成 Path 对象，后面就能方便地判断文件是否存在、读取和写入。

```
14         self.lock = RLock()
```

白话解释：创建可重入锁。多个线程可能同时读写 JSON 数据时，用它保证一次只有一个操作修改数据。

```
15         self.data: dict[str, Any] = {"sessions": {}, "user_memories": {}, "todos": {}}
```

白话解释：创建总数据字典。它有三块：sessions（短期上下文）、user_memories（长期记忆）、todos（待办）。

```
16         if self.path.exists():
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
17             self.data.update(json.loads(self.path.read_text(encoding="utf-8")))
```

白话解释：读取 JSON 文件里的文字，先 json.loads 转回 Python 字典，再 update 合并到内存中的 self.data。

```
18 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
19 def _save(self) -> None:
```

白话解释：私有辅助方法：把内存中的 data 写回 JSON 文件。前导下划线表示 ‘只给类内部使用’ 的约定。

```
20 self.path.parent.mkdir(parents=True, exist_ok=True)
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
21 self.path.write_text(json.dumps(self.data, ensure_ascii=False, indent=2), encoding="utf-8")
```

白话解释：json.dumps：把 Python 的字典/列表转换成 JSON 文本，方便写文件或作为 tool 消息传给模型。

```
22 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
23 def session(self, user_id: str, session_id: str) -> dict[str, Any]:
```

白话解释：根据 user_id 和 session_id 取出一个聊天窗口的数据；没有就创建一个。

```
24 # session 短期上下文的隔离边界：同一 user 的不同窗口也不会串数据。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
25 key = f"{user_id}:{session_id}"
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
26 with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
27 session = self.data["sessions"].setdefault(key, {"user_id": user_id, "session_id":  
> session_id, "summary": "", "messages": []})
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合 ‘没有 session 就新建 session’ 。

```
28 # Migration defaults for data written by v1.
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
29 session.setdefault("status", "idle")
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合 ‘没有 session 就新建 session’ 。

```
30 session.setdefault("pending_inputs", [])
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合 ‘没有 session 就新建 session’ 。

```
31 session.setdefault("events", [])
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合 ‘没有 session 就新建 session’ 。

```
32 return session
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
33 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
34 def save_session(self, session: dict[str, Any]) -> None:
```

白话解释：把一个 session 放回总数据并保存到硬盘。

```
35 with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
36 self.data["sessions"][f"{session['user_id']}:{session['session_id']}"] = session
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
37 self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
38 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
39 def claim_session(self, user_id: str, session_id: str, user_input: str) -> bool:
```

白话解释：尝试把 session 标记为 busy；若已忙，就把新消息放入 FIFO 队列。

```
40 """Claim an idle session; otherwise persist the new message in its FIFO queue."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、help() 等工具读取。

```
41 with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
42 session = self.session(user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
43 if session["status"] == "busy":
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
44 # FIFO 队列仅保存用户输入；执行时再按当时最新 Context 调用模型。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
45 session["pending_inputs"].append(user_input)
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
46 self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
47 return False
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
48 session["status"] = "busy"
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
49 self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
50 return True
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
51 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
52 def finish_session(self, user_id: str, session_id: str) -> None:
```

白话解释：把本轮请求结束的 session 改回 idle，让下一条排队消息可执行。

```
53 with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
54 session = self.session(user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
55 session["status"] = "idle"
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
56 self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
57 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
58 def claim_next_input(self, user_id: str, session_id: str) -> str | None:
```

白话解释：从等待队列取出最早的一条消息，并再次把 session 标记成 busy。

```
59     with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
60         session = self.session(user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
61         if session["status"] != "idle" or not session["pending_inputs"]:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
62             return None
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
63         session["status"] = "busy"
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
64         value = session["pending_inputs"].pop(0)
```

白话解释：pop(0)：取出并删除列表的第一个元素，所以等待队列遵守先进先出（FIFO）。

```
65         self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
66         return value
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
67 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
68     def add_event(self, user_id: str, session_id: str, event: dict[str, Any]) -> None:
```

白话解释：把异步任务的完成/失败/取消消息加入当前 session 的事件列表。

```
69         with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
70             self.session(user_id, session_id)["events"].append(event)
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
71             self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
72 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
73     def list_events(self, user_id: str, session_id: str, clear: bool = False) -> list[dict[str, Any]]:
```

白话解释：读取事件；clear=True 时读完同时清空。

```
74         with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
75             events = list(self.session(user_id, session_id)["events"])
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
76             if clear:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
77                 self.session(user_id, session_id)["events"] = []
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
78             self._save()
```


白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
79         return events
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
80 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
81     def remember(self, user_id: str, text: str) -> None:
```

白话解释：向用户级长期记忆写入一条信息。

```
82         # 长期记忆只按 user_id 索引，因此可以跨 session 共享。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
83         with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
84             memories = self.data["user_memories"].setdefault(user_id, [])
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合‘没有 session 就新建 session’。

```
85             if text not in memories:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
86                 memories.append(text)
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
87                 self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
88 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
89     def recall(self, user_id: str, query: str, top_k: int = 3) -> list[str]:
```

白话解释：从同一个用户的长期记忆中按简单匹配规则取 Top K 条。

```
90         # 第一/二版采用可解释的字符匹配；生产版可替换为 embedding 语义检索。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
91         characters = {ch.lower() for ch in query if not ch.isspace()}
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
92         memories = self.data["user_memories"].get(user_id, [])
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
93         ranked = sorted(memories, key=lambda item: sum(ch.lower() in characters for ch in item),
94                           > reverse=True)
```

白话解释：sorted：按给出的规则把列表排序。这里会让与用户当前问题更相关的长期记忆排在前面。

```
94         return ranked[:top_k]
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
95 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
96     def add_todo(self, user_id: str, session_id: str, task: str) -> dict[str, Any]:
```

白话解释：给某个聊天窗口添加待办。

```
97         with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
98         key = f"{user_id}:{session_id}"
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
99         todos = self.data["todos"].setdefault(key, [])
```

白话解释：setdefault：如果字典里已有这个键就取旧值；没有就创建默认值再返回。很适合‘没有 session 就新建 session’。

```
100         item = {"id": len(todos) + 1, "task": task, "done": False}
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
101         todos.append(item)
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
102         self._save()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
103         return item
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
104 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
105     def list_todos(self, user_id: str, session_id: str) -> list[dict[str, Any]]:
```

白话解释：列出某个聊天窗口的待办。

```
106         return self.data["todos"].get(f"{user_id}:{session_id}", [])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

tools.py

工具层。负责登记工具、检查参数并实际执行 calculator、todo 和异步任务工具。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 import ast
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 import operator
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 from dataclasses import dataclass
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
6 from typing import Any, Callable
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
7 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
8 from .store import JsonStore
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
9 from .tasks import AsyncTaskManager, delayed_search
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
10 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
11 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
12 @dataclass
```

白话解释：装饰器：让 Python 自动生成构造函数等常见方法。效果接近 Java 中简化 POJO 的工具。

```
13 class Tool:
```

白话解释：定义类 Tool。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
14     name: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
15     description: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
16     parameters: dict[str, Any]
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
17     handler: Callable[[dict[str, Any], dict[str, str]], Any]
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
18 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
19 def schema(self) -> dict[str, Any]:
```

白话解释：把工具自身描述转换为 DeepSeek function calling 需要的 JSON 格式。

```
20     return {"type": "function", "function": {"name": self.name, "description": self.description,  
        > "parameters": self.parameters}}
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
21 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
22 def validate(self, args: dict[str, Any]) -> None:
```

白话解释：在真正执行工具之前检查参数是否完整、枚举值是否有效。

```
23     # 在执行函数前拦截缺少必填项和 enum 非法值，错误可回传给模型修正。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
24     if not isinstance(args, dict):
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
25         raise ValueError("tool arguments must be an object")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
26     for field in self.parameters.get("required", []):
```

白话解释：循环：依次取出集合中的每个元素执行一次代码。类似 Java 的增强 for 循环。

```
27         if field not in args or args[field] in (None, ""):
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
28             raise ValueError(f"missing required argument: {field}")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
29     for field, spec in self.parameters.get("properties", {}).items():
```

白话解释：循环：依次取出集合中的每个元素执行一次代码。类似 Java 的增强 for 循环。

```
30         if field in args and "enum" in spec and args[field] not in spec["enum"]:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
31             raise ValueError(f"invalid {field}: {args[field]}")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
32 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
33 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
34 class ToolRegistry:
```

白话解释：定义类 ToolRegistry。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
35     def __init__(self) -> None:
```

白话解释：构造函数：创建对象时自动执行，用来准备这个对象后续工作需要的状态。

```
36     self._tools: dict[str, Tool] = {}
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
37 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
38     def register(self, tool: Tool) -> None:
```

白话解释：把一个工具放入 registry；同名工具不允许覆盖。

```
39     if tool.name in self._tools:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
40         raise ValueError(f"tool already registered: {tool.name}")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
41     self._tools[tool.name] = tool
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
42 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
43     def schemas(self) -> list[dict[str, Any]]:
```

白话解释：收集所有工具的 Schema，准备一起发给模型。

```
44         return [tool.schema() for tool in self._tools.values()]
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
45 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
46     def execute(self, name: str, args: dict[str, Any], context: dict[str, str]) -> Any:
```

白话解释：按工具名找到工具，先校验参数，再调用该工具的 handler。

```
47         if name not in self._tools:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
48             raise ValueError(f"unknown tool: {name}")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
49         tool = self._tools[name]
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
50         tool.validate(args)
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
51         return tool.handler(args, context)
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
52 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
53 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
54 _OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul, ast.Div: operator.truediv,
          > ast.USub: operator.neg}
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
55 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
56 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
57 def _calculate(expression: str) -> float:
```

白话解释：用 AST 白名单计算表达式，避免直接 eval 带来的任意代码执行风险。

58 # AST 白名单代替 eval, 禁止导入模块、属性访问和任意代码执行。

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

59 def visit(node: ast.AST) -> float:

白话解释: 定义函数 visit。括号里是输入参数, 箭头后的类型提示表示它预计返回什么。

60 if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):

白话解释: 条件判断: 条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

61 return node.value

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

62 if isinstance(node, ast.BinOp) and type(node.op) in _OPS:

白话解释: 条件判断: 条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

63 return _OPS[type(node.op)](visit(node.left), visit(node.right))

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

64 if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:

白话解释: 条件判断: 条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

65 return _OPS[type(node.op)](visit(node.operand))

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

66 raise ValueError("only +, -, *, / and parentheses are allowed")

白话解释: raise: 主动抛出错误, 告诉上层调用者输入不合法或当前状态不允许操作。

67 return visit(ast.parse(expression, mode="eval").body)

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

68 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

69 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

70 def build_default_registry(store: JsonStore, task_manager: AsyncTaskManager | None = None) ->
> ToolRegistry:

白话解释: 创建并注册本项目默认工具。

71 registry = ToolRegistry()

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

72 registry.register(Tool("calculator", "Evaluate a basic arithmetic expression.", {"type": "object",
> "properties": {"expression": {"type": "string"}}, "required": ["expression"]}, lambda a, _: {"result":
> _calculate(a["expression"]) })))

白话解释: 函数/方法调用: 点号前的对象调用某个能力; 括号里是传入的参数。

73 registry.register(Tool("search", "Search a mock knowledge base.", {"type": "object", "properties":
> {"query": {"type": "string"}}, "required": ["query"]}, lambda a, _: {"query": a["query"], "results":
> [f'Mock result for: {a["query"]}"]}))

白话解释: 函数/方法调用: 点号前的对象调用某个能力; 括号里是传入的参数。

74 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

75 def todo(args: dict[str, Any], ctx: dict[str, str]) -> Any:

白话解释: 定义函数 todo。括号里是输入参数, 箭头后的类型提示表示它预计返回什么。

76 if args["action"] == "add":

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
77         if not args.get("task"):
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
78             raise ValueError("task is required when action is add")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
79         return store.add_todo(ctx["user_id"], ctx["session_id"], args["task"])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
80         return store.list_todos(ctx["user_id"], ctx["session_id"])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
81     registry.register(Tool("todo", "Add or list todos belonging to the current session.", {"type":  
    > "object", "properties": {"action": {"type": "string", "enum": ["add", "list"]}, "task": {"type":  
    > "string"}}, "required": ["action"]}, todo))
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
82 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
83     if task_manager:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
84         # 只有传入任务管理器时才暴露异步工具，保持第一版核心的最小性。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
85         def background_search(args: dict[str, Any], ctx: dict[str, str]) -> Any:
```

白话解释：定义函数 background_search。括号里是输入参数，箭头后的类型提示表示它预计返回什么。

```
86             delay = float(args.get("delay_seconds", 0.2))
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
87             return task_manager.submit(ctx["user_id"], ctx["session_id"], "background_search", lambda  
    > cancel: delayed_search(args["query"], delay, cancel))
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
88     registry.register(Tool("background_search", "Start an asynchronous mock search; it returns a task  
    > id immediately.", {"type": "object", "properties": {"query": {"type": "string"}, "delay_seconds":  
    > {"type": "number"}}, "required": ["query"]}, background_search))
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
89 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
90     def task_control(args: dict[str, Any], ctx: dict[str, str]) -> Any:
```

白话解释：定义函数 task_control。括号里是输入参数，箭头后的类型提示表示它预计返回什么。

```
91         if args["action"] == "cancel":
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
92             return task_manager.cancel(args["task_id"], ctx["user_id"], ctx["session_id"])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
93             return task_manager.status(args["task_id"], ctx["user_id"], ctx["session_id"])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
94     registry.register(Tool("task_control", "Check or cancel an asynchronous task in the current  
    > session.", {"type": "object", "properties": {"action": {"type": "string", "enum": ["status", "cancel"]},  
    > "task_id": {"type": "string"}}, "required": ["action", "task_id"]}, task_control))
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

95 return registry

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

llm.py

模型通信层。负责调用 DeepSeek HTTP API、请求摘要、解析 function calling。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 import json
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 import os
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 import re
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
6 import urllib.error
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
7 import urllib.request
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
8 from typing import Any
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
9 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
10 from .types import LLMReply, ToolCall
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
11 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
12 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
13 class DeepSeekClient:
```

白话解释：定义类 DeepSeekClient。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
14     """Small OpenAI-compatible client; the Agent loop remains implemented locally."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、help() 等工具读取。

```
15 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
16     def __init__(self, api_key: str | None = None, base_url: str | None = None, model: str | None = None)
17     > -> None:
```

白话解释：构造函数：创建对象时自动执行，用来准备这个对象后续工作需要的状态。

```
17         self.api_key = api_key or os.getenv("DEEPSEEK_API_KEY")
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
18         self.base_url = (base_url or os.getenv("DEEPSEEK_BASE_URL",
```

```
> "https://api.deepseek.com")).rstrip("/")
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
19 self.model = model or os.getenv("DEEPSEEK_MODEL", "deepseek-chat")
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
20 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
21 def _request(self, messages: list[dict[str, Any]], tools: list[dict[str, Any]] | None = None) ->
    > dict[str, Any]:
```

白话解释：最底层网络请求：把 messages 和 tools 发送给 DeepSeek。

```
22 if not self.api_key:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
23 raise RuntimeError("Missing DEEPSEEK_API_KEY. Copy .env.example values into your
    > environment.")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
24 payload: dict[str, Any] = {"model": self.model, "messages": messages, "temperature": 0.2}
```

白话解释：创建网络请求需要的数据：payload 是要发送的 JSON；request 是包含地址、请求体和请求头的请求对象。

```
25 if tools is not None:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
26 # 传入工具 Schema 后，DeepSeek 才能返回 OpenAI-compatible tool_calls.
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
27 payload.update({"tools": tools, "tool_choice": "auto"})
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
28 request = urllib.request.Request(f"{self.base_url}/chat/completions",
    > data=json.dumps(payload).encode(), headers={"Authorization": f"Bearer {self.api_key}", "Content-Type":
    > "application/json"})
```

白话解释：json.dumps：把 Python 的字典/列表转换成 JSON 文本，方便写文件或作为 tool 消息传给模型。

```
29 try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
30 with urllib.request.urlopen(request, timeout=60) as response:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
31 return json.loads(response.read())
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
32 except urllib.error.HTTPError as exc:
```

白话解释：except：捕获 try 中发生的异常，避免整个程序直接终止。

```
33 raise RuntimeError(f"DeepSeek API failed ({exc.code}):
    > {exc.read().decode(errors='replace')}") from exc
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
34 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
35 def chat(self, messages: list[dict[str, Any]], tools: list[dict[str, Any]]) -> LLMReply:
```

白话解释：普通 Agent 对话调用：请求模型并把响应解析成 LLMReply。

```
36 return ToolCallParser.parse(self._request(messages, tools)["choices"][0]["message"])
```

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

37 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

38 def summarize(self, previous_summary: str, old_messages: list[dict[str, Any]]) -> str:

白话解释: 摘要调用: 让模型把旧聊天压缩成固定 JSON 结构。

39 # 摘要请求不传工具, 避免模型在压缩历史时触发业务工具调用。

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

40 source = [{"role": item["role"], "content": item.get("content", "")[:500]} for item in
> old_messages if item["role"] in {"user", "assistant", "tool"}]

白话解释: 列表推导式: 把 for 循环、条件判断和收集结果写在一行; 效果相当于创建空列表再循环 append。

41 prompt = ("Compress the conversation into JSON only. Required keys: goals, conclusions,
> open_items, preferences. "

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

42 "Each value must be a JSON array of concise strings. Preserve only durable task state;
> do not invent facts.\n"

白话解释: 普通表达式: 这一行会计算或访问一个值, 具体作用要结合上下文的变量名一起看。

43 f"Previous summary: {previous_summary}\nMessages: {json.dumps(source,
> ensure_ascii=False))}"

白话解释: json.dumps: 把 Python 的字典/列表转换成 JSON 文本, 方便写文件或作为 tool 消息传给模型。

44 content = self._request([{"role": "system", "content": "You produce valid JSON and nothing
> else."}, {"role": "user", "content": prompt}])["choices"][0]["message"].get("content", "{}")

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

45 return StructuredSummary.normalise(content)

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

46 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

47 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

48 class StructuredSummary:

白话解释: 定义类 StructuredSummary。类可以理解为 Java 的 class: 把数据和处理数据的方法组织在一起。

49 KEYS = ("goals", "conclusions", "open_items", "preferences")

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

50 <空行>

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

51 @classmethod

白话解释: 普通表达式: 这一行会计算或访问一个值, 具体作用要结合上下文的变量名一起看。

52 def normalise(cls, text: str) -> str:

白话解释: 检查并补齐摘要 JSON 的四个键, 保证 Runtime 使用时格式稳定。

53 # 校验并补齐 JSON 键, 保证 Runtime 后续拿到稳定的摘要结构。

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

54 clean = re.sub(r"^(?:json)?s*\s*`\$", "", text.strip(), flags=re.S)

白话解释: 用正则替换掉模型可能包上的 ``json 代码围栏, 只留下纯 JSON。

```
55     value = json.loads(clean)
```

白话解释: json.loads: 把 JSON 文本解析回 Python 的字典或列表。

```
56     if not isinstance(value, dict):
```

白话解释: 条件判断: 条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
57         raise ValueError("summary is not a JSON object")
```

白话解释: raise: 主动抛出错误, 告诉上层调用者输入不合法或当前状态不允许操作。

```
58     result = {key: [str(item) for item in value.get(key, [])[:8] for key in cls.KEYS]}
```

白话解释: 列表推导式: 把 for 循环、条件判断和收集结果写在一行; 效果相当于创建空列表再循环 append。

```
59     return json.dumps(result, ensure_ascii=False)
```

白话解释: return: 结束当前函数, 并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
60 <空行>
```

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

```
61 <空行>
```

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

```
62 class ToolCallParser:
```

白话解释: 定义类 ToolCallParser。类可以理解为 Java 的 class: 把数据和处理数据的方法组织在一起。

```
63     @staticmethod
```

白话解释: 普通表达式: 这一行会计算或访问一个值, 具体作用要结合上下文的变量名一起看。

```
64     def parse(message: dict[str, Any]) -> LLMReply:
```

白话解释: 从模型回复中取出原生 tool_calls; 也兼容 Markdown 里的 JSON 工具指令。

```
65     calls = []
```

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

```
66     for call in message.get("tool_calls") or []:
```

白话解释: 循环: 依次取出集合中的每个元素执行一次代码。类似 Java 的增强 for 循环。

```
67         # 标准 function calling: 解析模型给出的工具名和 JSON 参数。
```

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

```
68         fn = call["function"]
```

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

```
69         calls.append(ToolCall(call.get("id", f"call_{len(calls)}"), fn["name"],
> json.loads(fn.get("arguments") or "{}")))
```

白话解释: append: 把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
70     content = message.get("content") or ""
```

白话解释: 赋值: 先计算等号右边, 再把结果保存到等号左边的变量中。

```
71     if not calls:
```

白话解释: 条件判断: 条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
72         # 兼容模型偶尔以 Markdown JSON 而非原生 tool_calls 输出的情况。
```

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

```
73         match = re.search(r"``?(?json)?s*({.*?})s*``", content, re.S)
```

白话解释: 用正则是在用户输入中查找模式; 这里是在找用户是否明确说了‘记住……’。

```
74         if match:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
75         try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
76             item = json.loads(match.group(1))
```

白话解释：json.loads：把 JSON 文本解析回 Python 的字典或列表。

```
77             if "tool" in item:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
78                 calls.append(ToolCall("text_call_0", item["tool"], item.get("arguments", {})))
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
79         except json.JSONDecodeError:
```

白话解释：except：捕获 try 中发生的异常，避免整个程序直接终止。

```
80             pass
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
81     return LLMReply(content=content, tool_calls=calls)
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

tasks.py

异步任务层。负责后台执行耗时任务、取消任务，并把完成事件交回 session。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 from concurrent.futures import Future, ThreadPoolExecutor
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 from dataclasses import dataclass
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 from threading import Event, RLock
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
6 from time import sleep, time
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
7 from typing import Any, Callable
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
8 from uuid import uuid4
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
9 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
10 from .store import JsonStore
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
11 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
12 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
13 class TaskCancelled(Exception):
```

白话解释：定义类 TaskCancelled。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
14     pass
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
15 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
16 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
17 @dataclass
```

白话解释：装饰器：让 Python 自动生成构造函数等常见方法。效果接近 Java 中简化 POJO 的工具。

```
18 class AsyncTask:
```

白话解释：定义类 AsyncTask。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
19 task_id: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
20 user_id: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
21 session_id: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
22 name: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
23 status: str
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
24 cancel_event: Event
```

白话解释：普通表达式：这一行会计算或访问一个值，具体作用要结合上下文的变量名一起看。

```
25 future: Future | None = None
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
26 result: Any = None
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
27 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
28 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
29 class AsyncTaskManager:
```

白话解释：定义类 AsyncTaskManager。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
30 """In-process async task demo with completion events and cooperative cancellation."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、help() 等工具读取。

```
31 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
32 def __init__(self, store: JsonStore) -> None:
```

白话解释：构造函数：创建对象时自动执行，用来准备这个对象后续工作需要的状态。

```
33     self.store, self.lock = store, RLock()
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
34     self.executor = ThreadPoolExecutor(max_workers=2, thread_name_prefix="agent-task")
```

白话解释：创建线程池。它能让后台任务在线程中执行，而主 Agent 不必等待任务完成。

```
35     self.tasks: dict[str, AsyncTask] = {}
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
36 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
37 def submit(self, user_id: str, session_id: str, name: str, work: Callable[[Event], Any]) -> dict[str,  
    > Any]:
```

白话解释：创建一个异步任务并放入线程池，立刻返回 task_id。

```
38     # 任务立即返回 task_id，耗时 work 在线程池执行，不阻塞主 Agent Loop。
```


白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
39     task = AsyncTask(uuid4().hex[:10], user_id, session_id, name, "queued", Event())
```

白话解释：生成随机且几乎不会重复的 task_id，用于后续查询或取消具体任务。

```
40     with self.lock:
```

白话解释：with：进入一个受管理的代码块。这里用于加锁，离开缩进块后会自动释放锁。

```
41         self.tasks[task.task_id] = task
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
42 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
43     def runner() -> Any:
```

白话解释：定义函数 runner。括号里是输入参数，箭头后的类型提示表示它预计返回什么。

```
44         task.status = "running"
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
45         if task.cancel_event.is_set():
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
46             raise TaskCancelled()
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
47         return work(task.cancel_event)
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
48 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
49     def complete(future: Future) -> None:
```

白话解释：定义函数 complete。括号里是输入参数，箭头后的类型提示表示它预计返回什么。

```
50         # 后台任务结束后写入所属 session 的事件；下次交互可读取并通知用户。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
51         try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
52             task.result = future.result()
```

白话解释：取后台任务的最终结果；如果任务内部报错，这一行会重新抛出异常，交给 except 处理。

```
53             task.status = "completed"
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
54             event = {"type": "task_completed", "task_id": task.task_id, "name": name, "result":
> task.result}
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
55         except (TaskCancelled, Exception) as exc:
```

白话解释：except：捕获 try 中发生的异常，避免整个程序直接终止。

```
56             task.status = "cancelled" if isinstance(exc, TaskCancelled) or task.cancel_event.is_set()
> else "failed"
```

白话解释：isinstance：运行时检查一个值的类型。例如先确认模型传来的参数确实是字典。

```
57             event = {"type": f"task_{task.status}", "task_id": task.task_id, "name": name, "error":
> str(exc)}
```


白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
58     self.store.add_event(user_id, session_id, event)
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
59 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
60     task.future = self.executor.submit(runner)
```

白话解释：把 runner 函数提交给线程池，submit 会立刻返回 Future；任务在后台开始执行。

```
61     task.future.add_done_callback(complete)
```

白话解释：注册回调：后台任务无论成功、失败还是取消，结束时都会自动执行 complete。

```
62     return {"task_id": task.task_id, "status": "queued", "message": "Task started in background;
    > check task_control later."}
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
63 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
64     def cancel(self, task_id: str, user_id: str, session_id: str) -> dict[str, Any]:
```

白话解释：给指定任务发出取消信号。任务需要自己检查这个信号后退出。

```
65     # Python 不能安全强杀运行中的线程，所以采用 cancel_event 的协作式取消。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
66     task = self._owned(task_id, user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
67     task.cancel_event.set()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
68     if task.future:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
69         task.future.cancel()
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
70     return {"task_id": task_id, "status": "cancel_requested"}
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
71 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
72     def status(self, task_id: str, user_id: str, session_id: str) -> dict[str, Any]:
```

白话解释：查询某个任务当前状态和结果。

```
73     task = self._owned(task_id, user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
74     return {"task_id": task.task_id, "name": task.name, "status": task.status, "result": task.result}
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
75 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
76     def _owned(self, task_id: str, user_id: str, session_id: str) -> AsyncTask:
```

白话解释：校验 task_id 是否属于当前 user 和 session，防止串任务。

```
77     # task_id 即使猜中，也不能跨 user/session 查询或取消。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
78     task = self.tasks.get(task_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
79     if not task or (task.user_id, task.session_id) != (user_id, session_id):
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
80         raise ValueError("task not found in current session")
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
81     return task
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
82 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
83     def shutdown(self) -> None:
```

白话解释：关闭线程池，等待后台线程收尾。

```
84         self.executor.shutdown(wait=True)
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
85 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
86 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
87 def delayed_search(query: str, delay_seconds: float, cancel_event: Event) -> dict[str, Any]:
```

白话解释：可取消的模拟搜索：用短暂等待来演示长耗时网络任务。

```
88     # 用可取消的 mock 延迟模拟真实的长耗时网络工具。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
89     deadline = time() + max(0.05, min(delay_seconds, 2.0))
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
90     while time() < deadline:
```

白话解释：循环：只要条件为真就重复执行。这里通常会检查取消信号或等待任务完成。

```
91         if cancel_event.is_set():
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
92             raise TaskCancelled()
```

白话解释：raise：主动抛出错误，告诉上层调用者输入不合法或当前状态不允许操作。

```
93             sleep(0.02)
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
94     return {"query": query, "results": [f"Background mock result for: {query}"]}
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

runtime.py

最重要的 Agent 大脑。把模型、工具、记忆、上下文、队列连接成 Agent Loop。

逐行开始

```
1 from __future__ import annotations
```

白话解释：启用延迟解析类型标注。初学阶段可把它当成类型提示的兼容设置，不影响业务流程。

```
2 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
3 import json
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
4 import re
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
5 from threading import RLock
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
6 from typing import Any
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
7 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
8 from .store import JsonStore
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
9 from .tools import ToolRegistry
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
10 from .types import AgentResult
```

白话解释：导入：类似 Java 的 import。把别的模块、类或函数拿到当前文件使用。

```
11 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
12 SYSTEM_PROMPT = """You are a concise helpful assistant. Use a tool whenever it is needed. Do not invent  
> tool results. If a tool result answers the request, explain it to the user."""
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
13 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
14 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
15 class AgentRuntime:
```

白话解释：定义类 AgentRuntime。类可以理解为 Java 的 class：把数据和处理数据的方法组织在一起。

```
16     def __init__(self, client: Any, store: JsonStore, tools: ToolRegistry, max_turns: int = 5,  
> recent_messages: int = 10) -> None:
```

白话解释：构造函数：创建对象时自动执行，用来准备这个对象后续工作需要的状态。

```
17         self.client, self.store, self.tools = client, store, tools
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
18     self.max_turns, self.recent_messages = max_turns, recent_messages
```

白话解释：给当前对象保存一个属性。self 类似 Java 的 this，后续同一个对象的方法都能使用这个值。

```
19     self.lock = RLock()
```

白话解释：创建可重入锁。多个线程可能同时读写 JSON 数据时，用它保证一次只有一个操作修改数据。

```
20 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
21     def run(self, user_id: str, session_id: str, user_input: str) -> AgentResult:
```

白话解释：Agent 的对外入口：先处理 busy 状态，再开始本次请求。

```
22         # 先占用 session。若已被另一个请求占用，则只入队，不并发修改上下文。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
23         if not self.store.claim_session(user_id, session_id, user_input):
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
24             return AgentResult(session_id, "当前 session 正在处理上一条消息，新消息已进入队列。", [{"event": "input_queued",
> "input": user_input}])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
25         return self._run_claimed(user_id, session_id, user_input)
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
26 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
27     def drain_next(self, user_id: str, session_id: str) -> AgentResult | None:
```

白话解释：当前请求结束后，取出并执行一条排队消息。

```
28         """Process one queued user input after the active request has completed."""
```

白话解释：文档字符串：描述类或函数的用途，通常可被 IDE、help() 等工具读取。

```
29         queued = self.store.claim_next_input(user_id, session_id)
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
30         return self._run_claimed(user_id, session_id, queued) if queued is not None else None
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
31 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
32     def _run_claimed(self, user_id: str, session_id: str, user_input: str) -> AgentResult:
```

白话解释：真正的 Agent Loop：写入输入、组装上下文、调用模型、执行工具、直到得到最终回答。

```
33         session = self.store.session(user_id, session_id)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
34         trace: list[dict[str, Any]] = []
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
35         try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
36             # “记住……” 属于显式写入长期记忆，避免自动提取造成错误记忆。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
37             remembered = self._explicit_memory(user_id, user_input)
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
38         session["messages"].append({"role": "user", "content": user_input})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
39         self._compress(session, trace)
```

白话解释：检查历史是否过长；达到阈值时把旧消息压缩为摘要，避免 Context 无限增长。

```
40 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
41         for turn in range(1, self.max_turns + 1):
```

白话解释：循环：依次取出集合中的每个元素执行一次代码。类似 Java 的增强 for 循环。

```
42             # 每轮都重新组装 Context：记忆/摘要/最近消息会影响模型下一步决策。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
43             messages = self._context(session, user_id, user_input)
```

白话解释：调用 _context，把系统提示、长期记忆、摘要和最近聊天记录拼成这轮发给模型的 messages。

```
44             trace.append({"event": "llm_request", "turn": turn, "message_count": len(messages)})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
45             try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
46                 reply = self.client.chat(messages, self.tools.schemas())
```

白话解释：调用 llm.py 的客户端，请 DeepSeek 决定直接回答还是返回工具调用。

```
47             except Exception as exc:
```

白话解释：except：捕获 try 中发生的异常，避免整个程序直接终止。

```
48                 return AgentResult(session_id, f"模型调用失败：{exc}", trace + [{"event": "llm_error",  
                    > "error": str(exc)}])
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
49 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
50             if not reply.tool_calls:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
51                 # 没有工具调用即为最终回答，结束本轮 Agent Loop。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
52                 answer = reply.content or (f"我已记住：{remembered}" if remembered else "已完成。")
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
53                 session["messages"].append({"role": "assistant", "content": answer})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
54                 return AgentResult(session_id, answer, trace)
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
55 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
56         session["messages"].append({"role": "assistant", "content": reply.content or "",  
            > "tool_calls": [{"id": c.id, "type": "function", "function": {"name": c.name, "arguments":  
            > json.dumps(c.arguments, ensure_ascii=False)}} for c in reply.tool_calls]})
```

白话解释: append: 把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
57         for call in reply.tool_calls:
```

白话解释: 循环: 依次取出集合中的每个元素执行一次代码。类似 Java 的增强 for 循环。

```
58         try:
```

白话解释: try: 尝试执行可能失败的代码; 如果报错, 流程会跳到后面的 except。

```
59             # 工具异常会转成 tool result 回传给模型, 而不是让整个 Agent 崩溃。
```

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

```
60             result: Any = self.tools.execute(call.name, call.arguments, {"user_id": user_id,
> "session_id": session_id})
```

白话解释: 根据模型给出的工具名和参数, 调用 tools.py 中真正的本地工具。

```
61             trace.append({"event": "tool_success", "tool": call.name, "arguments":
> call.arguments, "result": result})
```

白话解释: append: 把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
62         except Exception as exc:
```

白话解释: except: 捕获 try 中发生的异常, 避免整个程序直接终止。

```
63             result = {"error": str(exc)}
```

白话解释: 把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
64             trace.append({"event": "tool_error", "tool": call.name, "error": str(exc)})
```

白话解释: append: 把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
65             session["messages"].append({"role": "tool", "tool_call_id": call.id, "content":
> json.dumps(result, ensure_ascii=False)})
```

白话解释: append: 把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
66             self._compress(session, trace)
```

白话解释: 检查历史是否过长; 达到阈值时把旧消息压缩为摘要, 避免 Context 无限增长。

```
67 <空行>
```

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

```
68         return AgentResult(session_id, "工具调用轮次已达到上限, 请缩小问题后重试。", trace, stopped_by_limit=True)
```

白话解释: return: 结束当前函数, 并把右边的值回调用它的地方。类似 Java/C 中的 return。

```
69         finally:
```

白话解释: finally: 无论 try 成功、失败或提前 return, 都会执行。适合做释放 session、保存数据等收尾工作。

```
70             # 无论成功、失败还是达到轮次上限, 都释放 session, 保证排队消息可继续执行。
```

白话解释: 注释: Python 会忽略这一行。它是写给人看的说明, 帮助你理解设计原因。

```
71             self.store.save_session(session)
```

白话解释: 函数/方法调用: 点号前的对象调用某个能力; 括号里是传入的参数。

```
72             self.store.finish_session(user_id, session_id)
```

白话解释: 函数/方法调用: 点号前的对象调用某个能力; 括号里是传入的参数。

```
73 <空行>
```

白话解释: 空行: 只用于把不同逻辑段落隔开, 不执行任何操作。

```
74     def _context(self, session: dict[str, Any], user_id: str, query: str) -> list[dict[str, Any]]:
```

白话解释: 按照系统提示、长期记忆、摘要、最近消息的顺序拼出发送给模型的 Context。

```
75         messages: list[dict[str, Any]] = [{"role": "system", "content": SYSTEM_PROMPT}]
```


白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
76     memories = self.store.recall(user_id, query)
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
77     if memories:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
78         messages.append({"role": "system", "content": "Relevant user memory (may be stale; current
> user instruction wins):\n- " + "\n- ".join(memories)})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
79     if session["summary"]:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
80         # 摘要为旧历史的压缩表示，最近消息仍保留原文以支持自然追问。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
81         messages.append({"role": "system", "content": "Structured session summary:\n" +
> session["summary"]})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
82     messages.extend(session["messages"][-self.recent_messages:])
```

白话解释：函数/方法调用：点号前的对象调用某个能力；括号里是传入的参数。

```
83     return messages
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
84 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
85     def _compress(self, session: dict[str, Any], trace: list[dict[str, Any]]) -> None:
```

白话解释：历史太长时，优先让模型输出结构化摘要；失败时使用文本兜底摘要。

```
86     messages = session["messages"]
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
87     if len(messages) <= self.recent_messages * 2:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
88         return
```

白话解释：return：直接结束当前函数，但不返回具体数据。

```
89     old, session["messages"] = messages[-self.recent_messages:], messages[-self.recent_messages:]
```

白话解释：赋值：先计算等号右边，再把结果保存到等号左边的变量中。

```
90     try:
```

白话解释：try：尝试执行可能失败的代码；如果报错，流程会跳到后面的 except。

```
91         # 第二版优先让模型输出四类结构化状态，避免普通摘要遗漏“未完成事项”。
```

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
92         session["summary"] = self.client.summarize(session["summary"], old)
```

白话解释：调用模型生成结构化摘要，专门保留目标、结论、未完成事项和偏好。

```
93         trace.append({"event": "structured_summary", "strategy": "llm_json"})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
94     except Exception as exc:
```

白话解释：except：捕获 try 中发生的异常，避免整个程序直接终止。

95 # API 不可用或 JSON 不合法时，使用确定性文本摘要作为兜底。

白话解释：注释：Python 会忽略这一行。它是写给人看的说明，帮助你理解设计原因。

```
96       facts = [f"{item['role']}: {item['content'][:160]}" for item in old if item["role"] in  
      > {"user", "assistant"} and item.get("content")]
```

白话解释：列表推导式：把 for 循环、条件判断和收集结果写在一行；效果相当于创建空列表再循环 append。

```
97       session["summary"] = (session["summary"] + "\n" + "\n".join(facts))[-2000:]
```

白话解释：把右边计算出的数据保存到左边变量。变量名提示了它保存的是会话、消息、记忆、任务或结果。

```
98       trace.append({"event": "structured_summary", "strategy": "fallback", "error": str(exc)})
```

白话解释：append：把一个元素追加到列表最后。这里用于添加消息、待办、事件或排队输入。

```
99 <空行>
```

白话解释：空行：只用于把不同逻辑段落隔开，不执行任何操作。

```
100   def _explicit_memory(self, user_id: str, text: str) -> str | None:
```

白话解释：只在用户明确说‘记住……’时写入长期记忆。

```
101       match = re.search(r'(?:::请)记住[: ;, \s]*(.+)', text)
```

白话解释：用正则用户在用户输入中查找模式；这里是在找用户是否明确说了‘记住……’。

```
102       if match:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
103           memory = match.group(1).strip("。！！")
```

白话解释：取正则第一个括号捕获到的内容，也就是‘记住’后面真正需要保存的文字；strip 会去掉首尾标点和空格。

```
104           if memory:
```

白话解释：条件判断：条件为真时才执行后面缩进的代码块。冒号后的缩进在 Python 中相当于 C/Java 的花括号。

```
105               self.store.remember(user_id, memory)
```

白话解释：调用 Store，把用户明确要求记住的信息写到 user_id 级长期记忆中。

```
106           return memory
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。

```
107       return None
```

白话解释：return：结束当前函数，并把右边的值交回调用它的地方。类似 Java/C 中的 return。